

NOTE:

- These are not exact midterm questions. These questions are intended to review the material.

1 Loop Invariants

Algorithm 1 Partition algorithm

```
1: function PARTITION( $A$ , low, high)
2:    $x \leftarrow 1$ 
3:    $y \leftarrow \text{length}(A)$ 
4:   for  $i \leftarrow 1$  to  $y + 1$  do
5:     if  $A[i] \leq \text{low}$  then
6:       SWAP( $A, i, x$ )                                ▷ Swaps  $A[i]$  and  $A[x]$ 
7:        $x \leftarrow x + 1$ 
8:     else if  $A[i] \geq \text{high}$  then
9:       SWAP( $A, i, y$ )                                ▷ Swaps  $A[i]$  and  $A[y]$ 
10:       $y \leftarrow y - 1$ 
11:       $i \leftarrow i - 1$ 
12:     end if
13:   end for
14: end function
```

1. What is the loop invariant?
2. Prove the loop invariant. Use the termination condition to prove the correctness of the original algorithm.

2 Reasoning with Short Justification

For each of the following statements, indicate whether the statement is true or false, and provide a brief justification.

- a. For two non-negative functions $f(n)$ and $g(n)$, if $f(n) = O(g(n))$, then $\log(f(n)) = O(\log(g(n)))$.
- b. If we permute an array of length n randomly (by selecting a permutation from any possible permutation uniformly at random), the average running time of INSERTION-SORT on the permuted array will be $\Theta(n \log n)$.
- c. Consider sorting the array $A = \langle 9, 5, 7, 6, 3, 4, 1, 2, 8 \rangle$ using RANDOMIZED-QUICKSORT. If we know that 5 and 7 are compared, but 5 and 8 are *not* compared during the course of the algorithm, we can conclude that 6 and 8 are not compared.

3 Create an Algorithm:

We are given an array D , which is an array of n distinct strings, sorted in lexicographic order. We are also given a procedure COMPARE-STRINGS(x, y), which will compare two strings x and y in $\Theta(1)$ time, and return true if x comes before y , and false if y comes before x in lexicographic order. We are also given an array C , which contains $n - 1$ of the n strings in D , but C is not sorted. We would like to develop an efficient algorithm that will find the missing string in C .